

Лабораторна робота № 1

Імперативна та структурна парадигми програмування

Дисципліна «Інженерія програмного забезпечення та мови програмування»

Спеціальність «Інженерія програмного забезпечення»

Перший курс

МЕТА РОБОТИ

Зрозуміти відмінність між неструктурованим імперативним керуванням і структурним програмуванням; навчитися аналізувати переходи `goto`, відновлювати логіку алгоритму та замінювати переходи послідовністю, розгалуженням і циклом без зміни поведінки програми.

РЕЗУЛЬТАТ РОБОТИ

Структурована версія індивідуальної програми мовою C без операторів `goto`; підтвердження однакового виведення початкової та переробленої програм; оформлений звіт.

ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ

Windows, Linux або macOS; Visual Studio Code чи інший редактор; компілятор C з підтримкою C11 (GCC або Clang). Початковий файл `task_XX_назва.c` надає викладач. Номер XX є номером індивідуального варіанта.

Викладач: Петров О.В., к.т.н., доцент.

ЗМІСТ

1. Підготовка та початковий запуск — с. 2.
2. Обов'язкове завдання — с. 3.
3. Перевірка та самоконтроль — с. 4.
4. Структура й оформлення звіту — с. 5–6.
5. Зразок титульного аркуша — с. 7.
6. Здавання та критерії перевірки — с. 8.
7. Установлення компілятора C у Windows — с. 9.

1. Підготовка до роботи

1.1. ОТРИМАЙТЕ СВІЙ ВАРІАНТ

Викладач призначає кожному студенту один файл із діапазону task_01_*.c—task_50_*.c. Працюйте лише зі своїм варіантом. Не обмінюйте початковий файл на інший після початку роботи.

1.2. СТВОРІТЬ РОБОЧУ ПАПКУ

Створіть папку Lab01_Прізвище_Група. Скопіюйте до неї призначений файл двічі та назвіть копії:

- task_XX_original.c — незмінний початковий код;
- task_XX_structured.c — файл для перероблення.

Відкрийте всю папку у VS Code командою **File** → **Open Folder**.

1.3. ПЕРЕВІРТЕ КОМПІЛЯТОР

Відкрийте **Terminal** → **New Terminal** і виконайте одну з команд:

```
gcc --version
```

```
clang --version
```

Достатньо одного доступного компілятора. Якщо обидві команди не розпізнаються, скористайтеся інструкцією для Windows на с. 9.

1.4. СКОМПІЛЮЙТЕ ПОЧАТКОВУ ПРОГРАМУ

Замініть XX своїм номером і виконайте:

```
gcc -std=c11 -Wall -Wextra -Wpedantic task_XX_original.c -o original
```

Якщо використовуєте Clang, замініть лише слово gcc на clang. Компіляція має завершитися без помилок і попереджень.

1.5. ЗАПУСТИТЬ І ЗБЕРЕЖІТЬ ЕТАЛОННЕ ВИВЕДЕННЯ

У Windows PowerShell виконайте:

```
.\original.exe | Tee-Object original_output.txt
```

У Linux або macOS виконайте:

```
./original | tee original_output.txt
```

Прочитайте виведення та коротко сформулюйте призначення програми. Надалі не змінюйте файл task_XX_original.c.

2. Обов'язкове завдання

2.1. ПРОАНАЛІЗУЙТЕ ПОЧАТКОВЕ КЕРУВАННЯ

Знайдіть усі мітки та оператори goto. Для кожної мітки визначте, звідки до неї переходить виконання і за якої умови. Виділіть:

- переходи вперед, що утворюють розгалуження;
- переходи назад, що утворюють цикли;
- переходи до завершення циклу або всієї програми;
- безумовні переходи та переходи всередині if.

Цей аналіз потрібен лише для планування перероблення; окремо оформлювати його для звіту не потрібно.

2.2. СПЛАНУЙТЕ СТРУКТУРНУ ЗАМІНУ

Для кожного фрагмента оберіть одну з базових структур:

- **послідовність** — команди виконуються одна за одною;
- **розгалуження** — if, if--else або вкладені умови;
- **повторення** — while, do--while або for;
- break чи continue, якщо вони безпосередньо передають зміст виходу з циклу або переходу до наступної ітерації.

2.3. ПЕРЕРОБІТЬ ПРОГРАМУ

Редагуйте лише task_XX_structured.c. Видаліть усі мітки та оператори goto, замінивши їх структурними конструкціями. Збережіть:

- початкові дані, числові константи та порядок їх опрацювання;
- формули, умови, накопичення значень і лічильники;
- тексти повідомлень та порядок виведення;
- усі особливі випадки: пропуск, відхилення, обмеження й завершення.

Можна вводити допоміжні логічні змінні та виділяти осмислені функції. Не можна підставляти наперед обчислену відповідь, вилучати гілки або імітувати goto змінною стану та великим switch.

2.4. ПРИВЕДІТЬ КОД ДО ЧИТАБЕЛЬНОГО ВИГЛЯДУ

Застосуйте однакові відступи, змістовні імена та короткі коментарі лише там, де причина дії неочевидна. Структура програми має читатися згори вниз.

ДОДАТКОВЕ ЗАВДАННЯ • НЕОБОВ'ЯЗКОВЕ

Винесіть один завершений етап алгоритму в окрему функцію та стисло поясніть, як це покращило структуру без зміни результату.

3. Перевірка результату

3.1. ПЕРЕВІРТЕ ВІДСУТНІСТЬ GOTO

Скористайтесь пошуком у файлі за словом `goto`. У фінальному коді не повинно залишитися ані операторів `goto`, ані міток, призначених для таких переходів. Переконайтеся, що цикли мають явні умови завершення.

3.2. СКОМПІЛЮЙТЕ СТРУКТУРНУ ВЕРСІЮ

```
gcc -std=c11 -Wall -Wextra -Wpedantic task_XX_structured.c -o structured
```

Виправте всі помилки й попередження. Потім збережіть результат запуску:

```
.\structured.exe | Tee-Object structured_output.txt
```

Для Linux або macOS використайте `./structured | tee structured_output.txt`.

3.3. ПОРІВНЯЙТЕ ВИВЕДЕННЯ

У Windows PowerShell виконайте:

```
Compare-Object (Get-Content original_output.txt) (Get-Content structured_output.txt)
```

У Linux або macOS виконайте:

```
diff -u original_output.txt structured_output.txt
```

За повного збігу обидві команди не виводять відмінностей. Сам факт успішної компіляції не доводить збереження поведінки.

3.4. ДОСЛІДІТЬ ОДНУ ПОМИЛКУ

У тимчасовій копії структурного файлу змініть умову завершення одного циклу так, щоб виникла зайва або пропущена ітерація. Скомпілюйте, порівняйте виведення, поясніть відмінність і видаліть тимчасову копію. Фінальний файл має залишитися правильним.

ЗАПИТАННЯ ДЛЯ САМОКОНТРОЛЮ

1. Які три базові керівні структури використовує структурне програмування?
2. Чому велика кількість довільних переходів ускладнює аналіз програми?
3. Як розпізнати цикл, організований переходом назад?
4. Коли під час перероблення доречні `break` і `continue`?
5. Чим `while` відрізняється від `do--while`?
6. Що означає поведінкова еквівалентність двох програм?
7. Чому одного тестового набору може бути недостатньо для доведення еквівалентності?

4. Структура звіту

1. ТИТУЛЬНИЙ АРКУШ

Оформіть за зразком на с. 7. Замініть поля у квадратних дужках власними даними. Назву дисципліни та тему лабораторної роботи не скорочуйте.

2. ТЕМА ТА МЕТА РОБОТИ

Укажіть номер і тему лабораторної роботи. Сформулюйте мету: переробити початкову програму без операторів `goto`, зберігши її логіку, структуру та результат роботи.

3. ВАРІАНТ І СЕРЕДОВИЩЕ

Укажіть номер та повне ім'я призначеного файлу, операційну систему, редактор і компілятор із версією. Версію отримайте командою `gcc --version` або `clang --version`.

4. ПОЧАТКОВА ПРОГРАМА

Наведіть повний текст `task_XX_original.c` та результат його виконання (вміст `original_output.txt`).

5. СТРУКТУРОВАНА ПРОГРАМА

Наведіть повний текст `task_XX_structured.c` та результат його виконання (вміст `structured_output.txt`). Код у звіті має точно відповідати електронному файлу.

5. Вимоги до оформлення

ФОРМАТ ДОКУМЕНТА

Звіт українською мовою. Формат А4, книжкова орієнтація. Поля: ліве 25 мм, праве 15 мм, верхнє й нижнє 20 мм. Основний текст: Times New Roman, 14 пт, міжрядковий інтервал 1,5; абзацний відступ 1,25 см.

ЗАГОЛОВКИ ТА СТОРІНКИ

Заголовки розділів — напівжирні, 14 пт. Титульний аркуш — окрема перша сторінка без видимого номера. Решту сторінок нумеруйте внизу, починаючи з 2. Фіксований обсяг не встановлюється: усі обов'язкові розділи мають бути змістовними й повними.

КОД ПРОГРАМИ

Код подавайте текстом моноширинним шрифтом Consolas або Courier New, 10–12 пт, з одинарним інтервалом і збереженими відступами. Не замінюйте код знімками екрана. Довгі рядки переносьте так, щоб вони не виходили за поля.

РЕЗУЛЬТАТИ ЗАПУСКУ

Вставте в звіт вміст `original_output.txt` і `structured_output.txt`. Виведення подавайте моноширинним шрифтом 10–12 пт.

ЗАХИСТ РОБОТИ

На захисті студент повинен показати у своєму варіанті принаймні один цикл, одне розгалуження та один вихід або пропуск, після чого пояснити їх перетворення.

ФІНАЛЬНИЙ ФОРМАТ

Підготуйте звіт у PDF для електронного подання. Якщо викладач вимагає паперовий примірник, роздрукуйте його на А4, перевірте читабельність коду, порядок сторінок і заповнення титульного аркуша.

Не додавайте до звіту сторонні варіанти, згенеровані відповіді без пояснення або фрагменти, яких немає у вашій фінальній програмі.

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Київський університет інтелектуальної власності та права

Кафедра кібербезпеки та інформаційних технологій

ЗВІТ

про виконання лабораторної роботи № 1

з дисципліни

**«Інженерія програмного забезпечення
та мови програмування»**

Тема: «Імперативна та структурна
парадигми програмування»

Виконав(ла):

студент(ка) 1 курсу, групи [шифр групи]

[Прізвище, ім'я, по батькові]

Варіант № [номер]

Перевірив:

Петров О.В., к.т.н., доцент

кафедри кібербезпеки та
інформаційних технологій

Київ, 2026

6. Здавання роботи

СКЛАД КОМПЛЕКТУ

У папці Lab01_Прізвище_Група мають бути:

- task_XX_original.c — незмінний файл викладача;
- task_XX_structured.c — структурована версія без goto;
- original_output.txt і structured_output.txt;
- Lab01_Прізвище_Група.pdf — оформлений звіт.

Перед надсиланням заархівуйте папку у ZIP, якщо цього вимагає обраний викладачем канал здавання. Будьте готові скомпілювати обидві програми та пояснити перетворення без читання готового тексту.

КРИТЕРІЇ ПЕРЕВІРКИ

Повнота: виконано перероблення, компіляцію, порівняння результатів та всі розділи звіту.

Правильність: структурна програма зберігає обчислення, гілки, кількість і порядок ітерацій та виводить той самий результат.

Структурність: у коді немає goto, прихованого програмного лічильника або великого switch, що відтворює початкові мітки.

Якість коду: конструкції вибрано відповідно до змісту алгоритму, відступи послідовні, імена та коментарі допомагають читанню.

Звіт і захист: електронні файли відповідають звіту, студент розуміє свій варіант і може пояснити основні рішення.

КОЛИ ПОТРІБНЕ ДООПРАЦЮВАННЯ

Програма не компілюється; залишилися оператори goto; результат або порядок повідомлень відрізняється; частину гілок вилучено; відповідь підставлено константою; структурне керування замінено машиною станів; початковий файл змінено; у звіті немає обох програм або результатів їх виконання.

ПЕРЕВІРТЕ ПЕРЕД ЗДАВАННЯМ

Номер варіанта та назви файлів правильні. Обидві програми компілюються з прапорцями попереджень. Виведення збігається. Пошук за словом goto у структурному файлі не дає результатів. Титульний аркуш заповнено. У ZIP немає виконуваних файлів, тимчасових копій або чужих варіантів.

Додаткове завдання не замінює обов'язкових пунктів. На захисті покажіть один цикл і одне розгалуження до та після перетворення.

7. Установлення GCC

1. УСТАНОВІТЬ MSYS2

Відкрийте офіційну сторінку <https://www.msys2.org/>, завантажте інсталятор і виконайте стандартне встановлення. Після завершення відкрийте середовище **MSYS2 UCRT64**.

2. ОНОВІТЬ СИСТЕМУ ПАКЕТІВ

У терміналі MSYS2 UCRT64 виконайте:

```
pacman -Syu
```

Якщо термінал попросить завершити роботу, закрийте його, знову відкрийте MSYS2 UCRT64 і повторіть команду.

3. УСТАНОВІТЬ НАБІР ІНСТРУМЕНТІВ

```
pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
```

Підтвердьте запропонований набір пакетів. Він містить GCC та інші засоби, необхідні для компіляції й налагодження.

4. ДОДАЙТЕ GCC ДО PATH

Додайте каталог C:\msys64\ucrt64\bin до системної або користувацької змінної Path. Повністю закрийте й знову відкрийте VS Code. У новому терміналі перевірте:

```
gcc --version
```

5. НАЛАШТУЙТЕ VS CODE

У розділі Extensions установіть розширення **C/C++** від Microsoft. Редактор і розширення не містять власного компілятора: команда gcc має працювати в терміналі незалежно від кнопки запуску VS Code.

ЯКЩО КОМПІЛЯЦІЯ НЕ ВДАЄТЬСЯ

Якщо gcc не розпізнається, перевірте шлях C:\msys64\ucrt64\bin і перезапустіть VS Code. Якщо компілятор не знаходить файл, перевірте поточну папку командами `pwd` і `ls` у MSYS2 або `Get-Location` і `Get-ChildItem` у PowerShell.

ОФІЦІЙНІ ДЖЕРЕЛА

MSYS2: <https://www.msys2.org/docs/installer/>

Visual Studio Code: <https://code.visualstudio.com/docs/cpp/config-mingw>